

EE523 - Inertial Navigation Systems
Project 3 - Report

Kléber Macedo Cabral

October 2018

Contents

1	Introduction	3
2	Related Works	4
3	System Design	5
3.1	Taylor Series trilateration algorithm	6
3.2	Kalman Filter	7
4	Results and Discussion	10
4.1	Flight scenario 1	11
4.2	Flight scenario 2	14
4.3	Flight scenario 3	15
4.4	Flight scenario 4	18
5	Conclusion	19

List of Figures

3.1	System architecture	5
3.2	Intersection of range measurements.	6
4.1	Simulation architecture.	10
4.2	Simulated quadrotor.	11
4.3	Flight configuration.	11
4.4	Full trajectory of the <i>mobile target</i> UAV.	12
4.5	Position estimation of the <i>mobile target</i> during ascension (movement in the Z_I axis).	12
4.6	Position estimation of the <i>mobile target</i> during hovering.	13
4.7	Position estimation of the <i>mobile target</i> during translation.	13
4.8	P matrix - Kalman filter.	14
4.9	Position during the translation of the <i>mobile references</i>	15
4.10	Position during the translation of all vehicles.	15
4.11	Flight outside the flight area.	16
4.12	Position estimation for different arrangements of the <i>mobile references</i>	17
4.13	Flight configurations.	17
4.14	Full trajectory of the <i>mobile target</i> UAV.	18

Chapter 1

Introduction

There are many approaches to identify the position of a moving robot in indoor and outdoor environments. For instance, the GNSS are systems that use signals from satellites around the Earth to calculate the position of a receiver module on the surface of the planet.

Moreover, systems such as OptiTrack (www.optitrack.com) are a possibility for indoor spaces. They rely on multiple infrared cameras to capture images of the environment. Then, through the computation of those images it is possible to calculate the position of an object in space.

However, errors are inherent to measurements and estimations performed by any sensor. Therefore, I investigate in this work how to calculate the position of robots in a multi-agent system, using the range measurements between quadrotors to achieve a better performance. Such approach can be categorized as *Cooperative Localization*.

In the approach presented in this work, one of the quadrotors has a accelerometer device coupled to its frame, and it is able to measure its distance to other UAVs (Unmanned Autonomous Vehicles, UAVs) during the flight. Also, a trilateration algorithm is used to calculate a possible three-dimensional position from the range estimations, and a Kalman Filter is used to fuse this position estimation with the accelerometer measurements.

There will be presented the results from simulation trials, where the performance of the proposed estimation technique is evaluated.

This report is divided as follows: Chapter 2 provides a short literature review of the Cooperative Localization problem. Chapter 3 depicts the implementation of the estimation technique and explains the Kalman Filter and Trilateration Algorithm used. Chapter 4 presents the results and the related discussion. Finally, Section 5 presents the conclusion.

Chapter 2

Related Works

During this chapter, some works regarding the Cooperative Localization problem will be presented. The main goal of this literature review is to understand: a) what are the techniques involved in the estimation problem, and b) what are the mathematical models used to describe the states in each system (movement model, measurements model, etc).

Bounini *et al.* [1] proposed a system where the position of land vehicles was computed using two non-linear models: a) vehicle motion model, and b) relative distance and bearing angle between two vehicles. The work presented simulated results where an Extended Kalman Filter estimated the position and velocity of each vehicle. A simulation showed that the efficiency of the system increased as the number of vehicles in operation increased.

Kurazume *et al.* [2] studied the cooperative positioning of vehicles using four approaches: 1) two robots that measure distances and angles between them; 2) three robots that measure only the angles; 3) three robots that measure the distances and angles; and 4) larger number of robots that measure distances and angles.

The agents used only relative information to calculate its actual position, that is, the computation of a new position depends on: a) the vehicle previous position, and b) the position of the other robot(s) at that instant. Using a Monte Carlo simulation the authors showed that the covariance of the position estimation grows as the number of the movements performed by the robots grows.

The cooperative localization can be interesting in a GPS-denied environment, for example, underwater. Therefore, some works studied the problem for Autonomous Underwater Vehicles (AUVs). In [3], the authors present a system that uses acoustic ranges from multiple surface vehicles to calculate the position and orientation of a submerged robot. The authors also presented a Monte Carlo approach based on: a) an EKF to estimate a more accurate range between submerged and surface vehicles, and b) a Particle Filter to calculate the pose of the underwater robot.

In [4], the authors also explored the localization of AUVs using the ranging between surface and underwater robots. Although, instead of using Bayesian estimation (as in a Kalman Filter), the authors proposed an approach that uses a cost function to determine the most probable transition between two states (positions) of the robot.

Tanwar *et al.* [5] presented a study using multiple Unmanned Aerial Vehicles (UAVs). The vehicles were able to communicate and exchange: a) GPS pseudoranges, b) inter-agent range information, c) their calculated states and covariances, and d) cross-correlations (covariances between robots). It was used a decentralized cooperative localization approach. Therefore each vehicle computed its position and covariances, and the cross covariances of the inter-agent ranging measurements.

Generally, the works presented have in common three characteristics: 1) mathematical models for the motion of the vehicles and the measurements performed, 2) the use of an additional measurement from exteroceptive sensors (such as range sensors), 3) the presence of a state estimator capable of integrating multiple measurements. Thus, the system proposed on this work will be based on the same characteristics in order to estimate the position of an UAV.

Chapter 3

System Design

This work presents a centralized approach for cooperative localization of quadrotors. The UAVs are divided into two categories: 1) the *mobile references* and 2) the *mobile target*. The *mobile references* are vehicles that have a very accurate position information available. The *mobile target* is the quadrotor that we intend to estimate the position.

It is considered that the *mobile target* is capable of measuring the distances between itself and the other UAVs through some kind of range sensor (ultrasonic, LIDAR, etc.). Also, the *mobile target* is embedded with a 3-axes accelerometer, that provides the actual acceleration of the vehicle in the body frame.

In this work, it is proposed the system architecture showed in figure 3.1, in order to estimate the position of the *mobile target*. Note that the range measurements are used in a trilateration algorithm, generating a first estimation of the three-dimensional position of the vehicle. This estimation is integrated with the measurements from a embedded accelerometer in a Kalman Filter, in order to compute a more accurate position of the UAV.

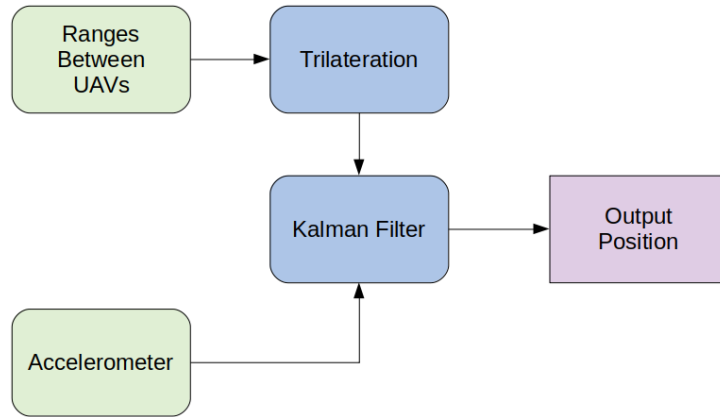


Figure 3.1: System architecture

A inertial coordinate frame S_I can be described, fixed in the environment. The X_I and Y_I axes describe the horizontal plane at the ground level, and the Z_I axis points upwards. Additionally, a coordinate system located at the center of mass of each the quadrotor (S_q) can be described. The X_q and Y_q axes describe the plane of the propellers, with X_q point to the front of the vehicle, and Y_q pointing to the left of the vehicle. Also, the Z_q axis is perpendicular to X_q and Y_q and points upwards.

The position of the quadrotor, $p = [x \ y \ z]^T$, describes the origin of S_q with respect to S_I . The orientation of the quadrotor is given by the Euler angles of pitch (θ), roll (ϕ), and yaw (ψ).

For simplicity, in this work the positions of the *mobile references* are known and accurate. Therefore, it is not modeled here the cross covariance between position estimation of the vehicles (as in [5] and [2]).

The section 3.1 will explain the trilateration algorithm used, and the section 3.2 will explain the Kalman Filter.

3.1 Taylor Series trilateration algorithm

In order to calculate the position of the *mobile target* ($p = [x \ y \ z]^T$), it is used the Taylor Series algorithm presented in [6]. Figure 3.2 shows the intersection region between multiple range estimations. The trilateration algorithm computes the point inside the region that best describes this intersection.

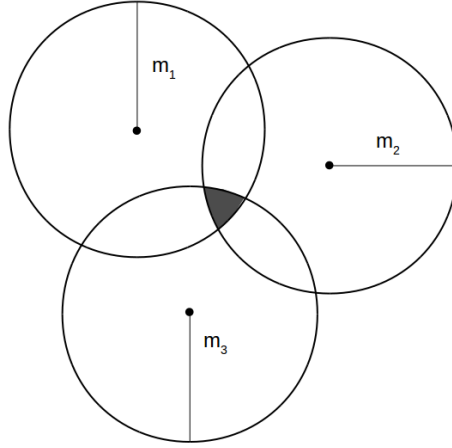


Figure 3.2: Intersection of range measurements.

We can consider a function f_i as being a Taylor series approximation of the distance measurement (m_i) between any *mobile reference* i and the *mobile target*. Thus, f_i is given by the equations (3.1).

$$f_i(x, y, z) = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} = m_i + \varepsilon_i \quad (3.1)$$

where ε_i is the measurement error and ($i = 1, 2, \dots, n$), and n is the number of range measurements. This error can be described by a Gaussian probability distribution function with zero mean and variance σ_i . Thus, we can also define the Q matrix as:

$$\mathbf{Q} = \text{diag} [\sigma_1^2 \quad \sigma_2^2 \quad \dots \quad \sigma_n^2] \quad (3.2)$$

Given an initial estimation of the position of the mobile module $p_{\text{initial}} = (x_v, y_v, z_v)$, it is possible to calculate:

$$x = x_v + \delta_x \quad (3.3)$$

$$y = y_v + \delta_y \quad (3.4)$$

$$z = z_v + \delta_z \quad (3.5)$$

where δ_x , δ_y e δ_z are the errors between the initial estimation of position and the correct position. We can denominate the vector $\vec{\delta} = [\delta_x \ \delta_y \ \delta_z]^T$ as the residue vector.

Expanding the equation (3.1) in a first order Taylor series, evaluated at (x_v, y_v, z_v) , we have:

$$f_i(x, y, z) \approx f_i(x_v, y_v, z_v) + \left. \frac{\delta f}{\delta x} \right|_{(x_v, y_v, z_v)} (x - x_v) + \left. \frac{\delta f}{\delta y} \right|_{(x_v, y_v, z_v)} (y - y_v) + \left. \frac{\delta f}{\delta z} \right|_{(x_v, y_v, z_v)} (z - z_v) \quad (3.6)$$

where $f = f_i(x, y, z)$ from equation (3.1).

Let us simplify the notation using $f_i(x_v, y_v, z_v) = f_{i,v}$. Also, using the equations (3.3) to (3.5), we can write:

$$f_i(x, y, z) \approx f_{i,v} + a_{i,x}\delta_x + a_{i,y}\delta_y + a_{i,z}\delta_z \approx m_i + \varepsilon_i \quad (3.7)$$

The coefficients $a_{i,x}$, $a_{i,y}$ e $a_{i,z}$ are obtained through the partial derivative of $f(x, y, z)$ (equation (3.1)) by x , y e z . Thus:

$$a_{i,x} = \left. \frac{\delta f}{\delta x} \right|_{(x_v, y_v, z_v)} = \frac{x_v - x_i}{r_v} \quad (3.8)$$

$$a_{i,y} = \left. \frac{\delta f}{\delta y} \right|_{(x_v, y_v, z_v)} = \frac{y_v - y_i}{r_v} \quad (3.9)$$

$$a_{i,z} = \left. \frac{\delta f}{\delta z} \right|_{(x_v, y_v, z_v)} = \frac{z_v - z_i}{r_v} \quad (3.10)$$

where,

$$r_v = \sqrt{(x_v - x_i)^2 + (y_v - y_i)^2 + (z_v - z_i)^2} \quad (3.11)$$

Since $i = 1, 2, \dots, n$, we can expand the equation (3.7) into the matrices. Thus, we obtain:

$$\mathbf{A} \vec{\delta} = \vec{D} + \vec{e} \quad (3.12)$$

where

$$\mathbf{A} = \begin{bmatrix} a_{1,x} & a_{1,y} & a_{1,z} \\ a_{2,x} & a_{2,y} & a_{2,z} \\ \dots & \dots & \dots \\ a_{n,x} & a_{n,y} & a_{n,z} \end{bmatrix} \quad (3.13)$$

$$\vec{D} = [(m_1 - f_{1,v}) \ (m_2 - f_{2,v}) \ \dots \ (m_n - f_{n,v})]^T \quad (3.14)$$

$$\vec{e} = [\varepsilon_1 \ \varepsilon_2 \ \dots \ \varepsilon_n]^T \quad (3.15)$$

Lastly, the solution for this equation system is:

$$\vec{\delta} = (\mathbf{A}^T \mathbf{Q}^{-1} \mathbf{A})^{-1} \mathbf{A}^T \mathbf{Q}^{-1} \vec{D} \quad (3.16)$$

After computing the residue vector ($\vec{\delta}$), it is possible to obtain a more accurate estimation of the position, given the equations (3.3) to (3.5). Thus, p is a function of the initial estimation and the ranges measured, $p = f(p_{initial}, m_i)$.

The estimation of the position, $p = [x \ y \ z]^T$, can also be reinserted in the algorithm as an initial estimation, $p_{initial} = p$. This process can be done multiple times, until the difference between the **initial estimation** and the **position computed** is sufficiently small ($p_{initial} \approx p$), indicating the convergence of the algorithm.

3.2 Kalman Filter

A Kalman Filter is used to estimate the physical states of the vehicle, that is, position and velocity in X_I , Y_I , and Z_I . The state vector is $x = [p_x \ p_y \ p_z \ v_x \ v_y \ v_z]^T$, and a kinematic model is used to compute the motion of the UAV:

$$x_{k+1} = Fx_k + Gu \quad (3.17)$$

$$y_k = Hx_k \quad (3.18)$$

$$F = \begin{bmatrix} 0_3 & I_3 \\ 0_3 & 0_3 \end{bmatrix} \quad (3.19)$$

$$G = \begin{bmatrix} 0_3 \\ I_3 \end{bmatrix} \quad (3.20)$$

$$u = [a_x^I \quad a_y^I \quad a_z^I]^T \quad (3.21)$$

$$H = [I_3 \quad 0_3] \quad (3.22)$$

where I_3 is the identity matrix and 0_3 is a matrix with all elements equal to zero, both matrices with size 3 by 3. k is the epoch (iteration step) of the algorithm.

The measurements from the accelerometer are described in the body frame of the UAV (S_q). Thus, in order to obtain the accelerations in the inertial frame (u vector, in S_I), it is necessary to rotate the accelerometer measurements using the equation:

$$\begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}^I = \begin{bmatrix} c\theta c\psi & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ -s\theta & s\phi c\theta & c\phi c\theta \end{bmatrix} \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}^q + \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} \quad (3.23)$$

where $c(\cdot)$ and $s(\cdot)$ denote *cosine* ($\cdot$) and *sine* ($\cdot$), respectively. Also, g is the gravity value.

For simplicity, it is assumed that the attitude angles (ϕ, θ, ψ) are the ideal values, without measurement errors. In the future, I intend to improve this approach and include the error dynamics in the model.

The accelerometer measurements can be considered as a combination of the real acceleration and a Gaussian noise with *mean* $\neq 0$ (bias). Therefore:

$$a_x^q = a_x^{real} + bias_{ax} + \eta_{ax} \quad (3.24)$$

additionally, the same can be stated for a_y^q, a_z^q . Note that, given the accelerometer errors described, one can assume that the estimation performed by the kinematic model only will diverge in the long run.

The Kalman Filter has three stages.

1. The *Initialization* stage:

- Initialize the variables P, Q, R;
- Initialize the state vector x .

2. The *Prediction* stage: Utilizes the kinematic model to estimate the position and velocity of the quadrotor.

$$Q_d = (GQG^T)\Delta t \quad (3.25)$$

$$\Phi = I + F\Delta t \quad (3.26)$$

$$x = \Phi x + (G\Delta t)u \quad (3.27)$$

$$P = \Phi P \Phi^T + Q_d \quad (3.28)$$

$$P = (P^T + P)/2 \quad (3.29)$$

3. The *Update* stage: Uses the trilateration estimation ($Z = [x \ y \ z]^T$), calculated through the distances between UAVs, to improve the position estimation performed by the kinematic model (vector x).

$$K = PH^T(HPH^T + R)^{-1} \quad (3.30)$$

$$x = x + K(Z - Hx) \quad (3.31)$$

$$P = (I - KH)P(I - KH)^T + KRK^T \quad (3.32)$$

where:

- P is the covariance matrix;
- F and G are the kinematic model matrices;
- Φ is the state transition matrix;
- x is the state vector;
- u is a vector with the accelerometer measurements;
- Δt is the time step of the algorithm (0.1 second in this work);
- Z is the vector with the position coordinates calculated with the trilateration algorithm;
- K is the Kalman Gain;
- H is the design matrix;
- Q is the system noise matrix (accelerometer noise, η_{ax} , η_{ay} , η_{az});
- R is the measurement noise matrix (trilateration noise).

During the execution of the algorithm, numeric errors can make the P matrix stop being symmetric. To correct that, it was implemented the equation $P = (P^T + P)/2$ after the prediction stage of the algorithm. This equation *forces* P back to symmetry.

Chapter 4

Results and Discussion

The simulation architecture is shown in figure 4.1. All the controllers of the *mobile references* used the real values (ideal values, output of the simulation). On the other hand, the position controller of the *mobile target* used the estimated value, output of the Kalman Filter.

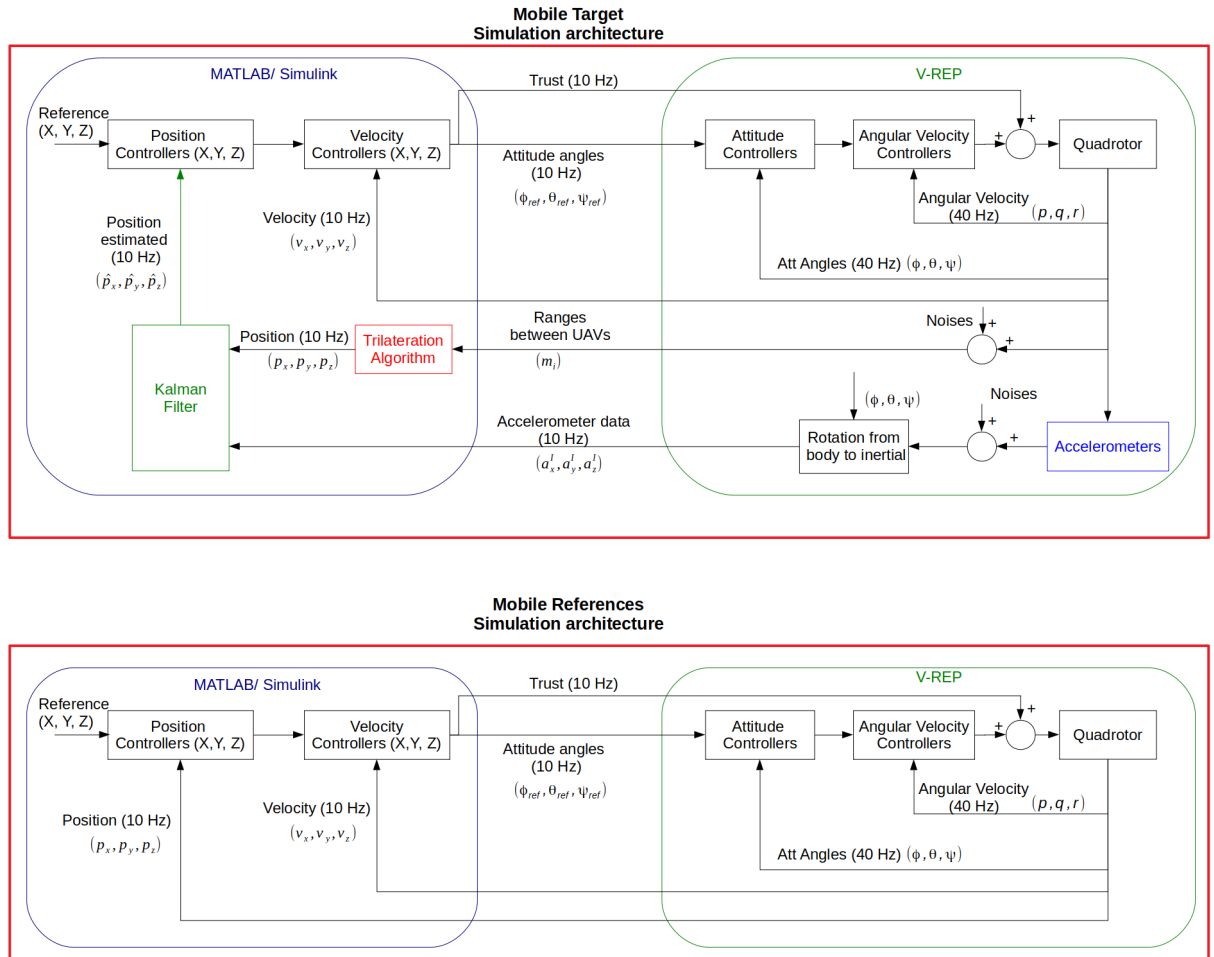


Figure 4.1: Simulation architecture.

The range measurements between UAVs were simulated by adding a Gaussian noise to the ideal

Table 4.1: Gaussian noises added to the simulated measurements.

	Mean [m]	Variance [m]
Accelerometer	± 0.01	0.0001
Range	0	0.0025

distances between the quadrotors. The added noises are shown in the table 4.1. Also, the bias value (mean) added to the accelerometer measurements randomly changed its signal every 20 seconds.

Figure 4.2 shows the quadrotor used in the simulation. It is a 3D model of the quadrotors called BitDrones, presented in [7].

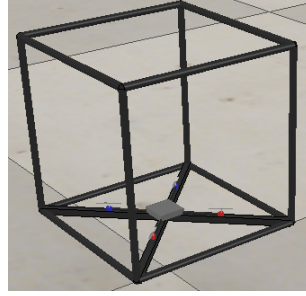


Figure 4.2: Simulated quadrotor.

4.1 Flight scenario 1

During the first simulation trial, the quadrotors that act as *mobile references* performed hover flight at the positions described in the table 4.2. The *mobile target* maneuvered inside a virtual triangle formed by the *mobile references* in the horizontal plane, and also with a lower height (figure 4.3). Later in this chapter, flight outside this area will be considered.

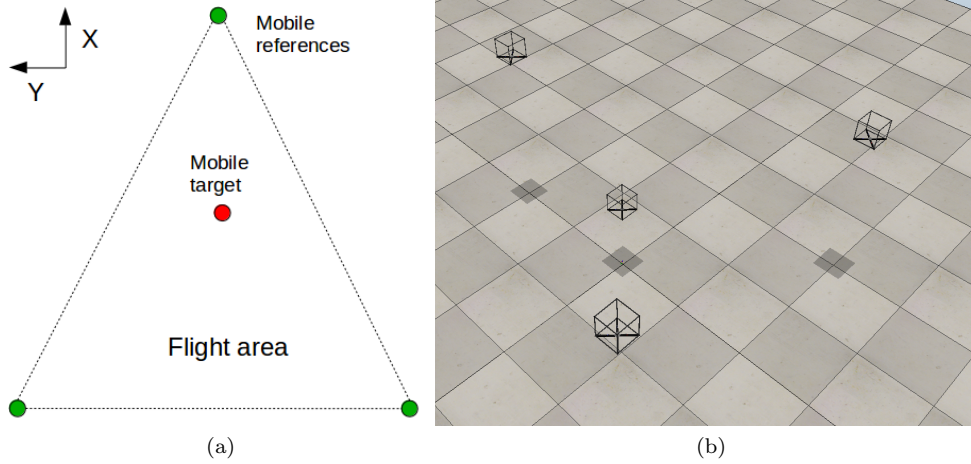


Figure 4.3: Flight configuration.

The *mobile target* performed three maneuvers: a) ascension (figure 4.5), b) hovering (figure 4.6), and c) translation in X_I , Y_I , and Z_I (figure 4.7). Additionally, the figure 4.4 shows the whole simulation.

Table 4.2: Coordinates for the hover flight of the UAVs used as references

	X [m]	Y [m]	Z [m]
Mobile reference 1	-1.00	-1.00	1.05
Mobile reference 2	-1.00	1.00	1.10
Mobile reference 3	1.00	0.00	1.15

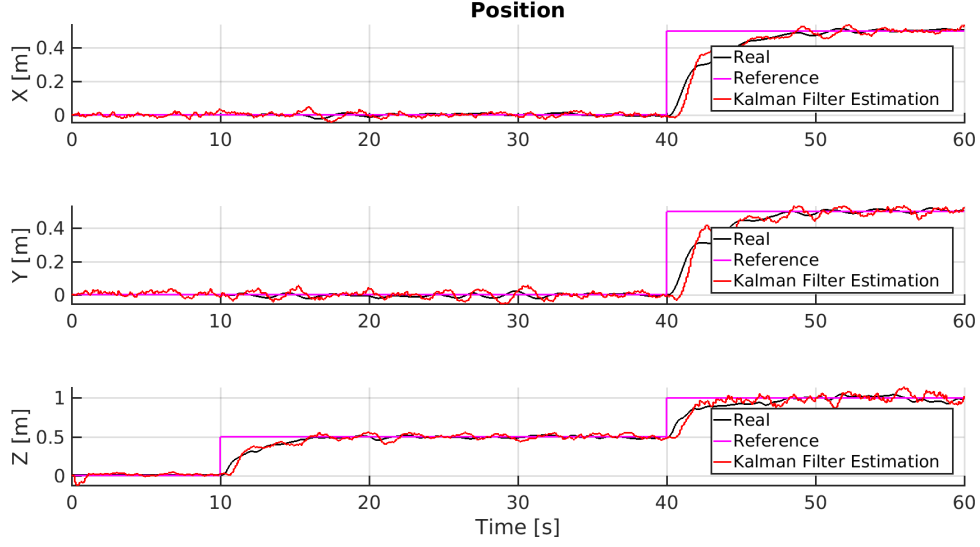


Figure 4.4: Full trajectory of the *mobile target* UAV.

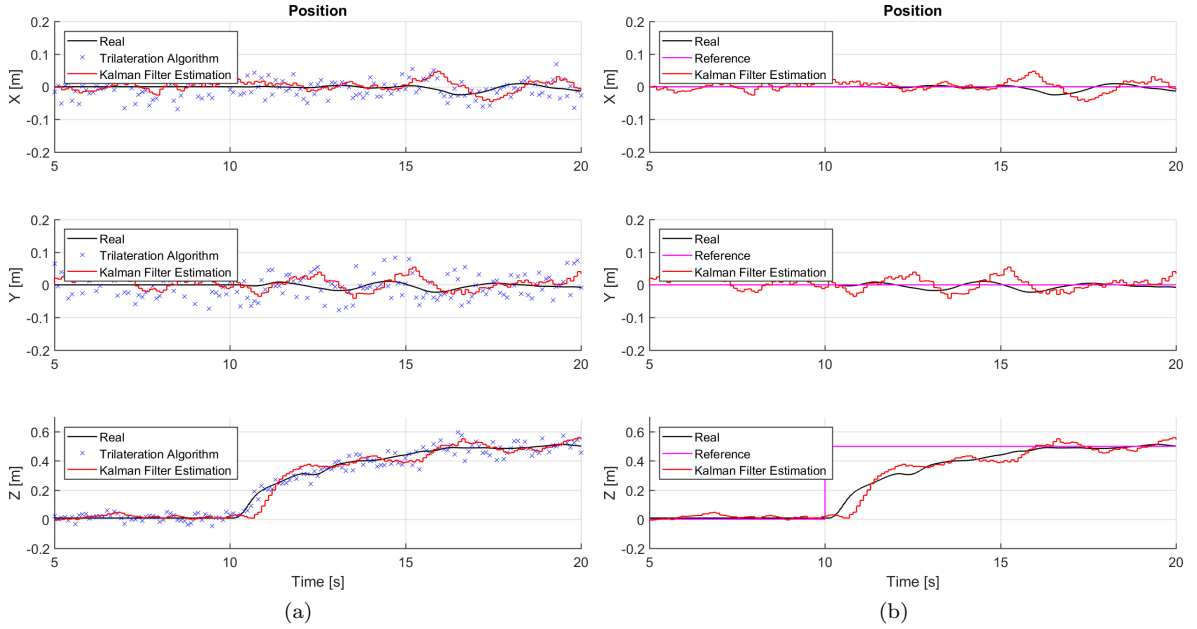


Figure 4.5: Position estimation of the *mobile target* during ascension (movement in the Z_I axis).

The RMSE (root mean squared error) of the position estimation during the three maneuvers is presented in the table 4.3. It is possible to see that the estimation has a centimeter level error. The

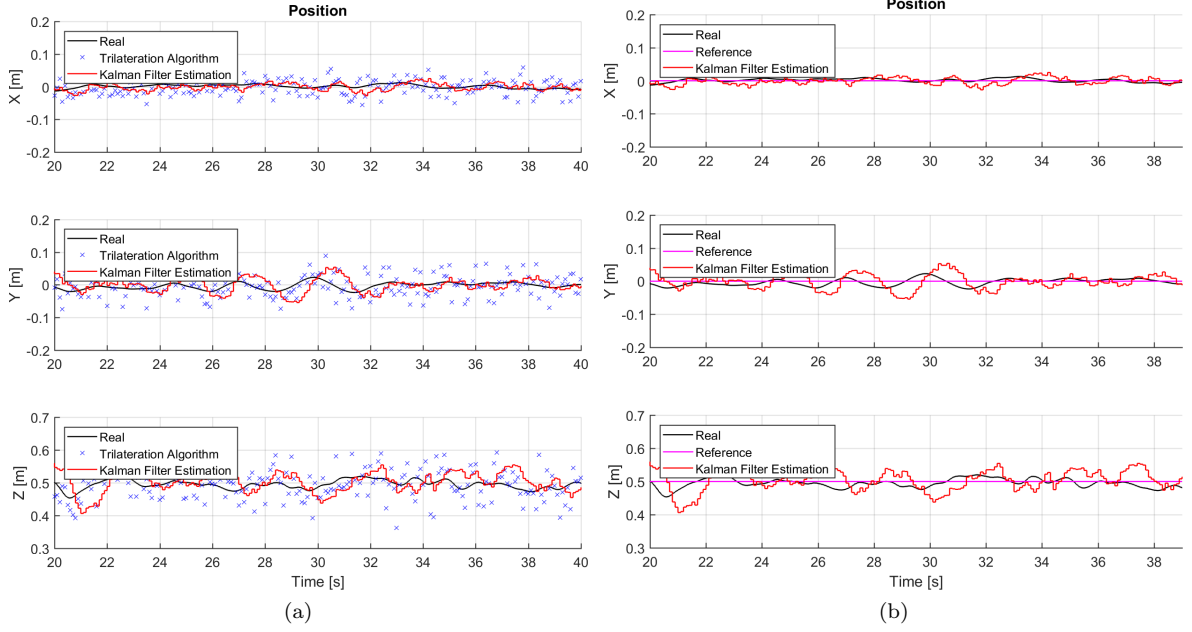


Figure 4.6: Position estimation of the *mobile target* during hovering.

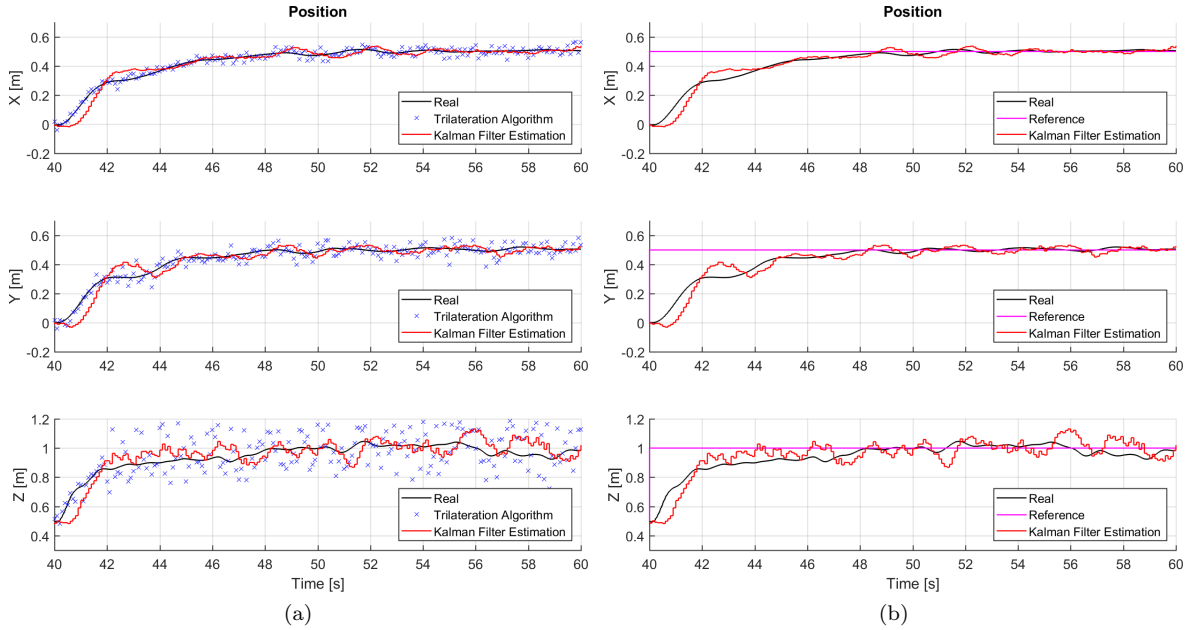


Figure 4.7: Position estimation of the *mobile target* during translation.

relatively low value of the error allows the estimation to be used in the position controller.

If we analyze the output of the trilateration algorithm only, it is possible to see that the uncertainty of the height estimation increased as the *mobile target* approached the height of the other vehicles (table 4.4).

Note that, for this experiment, the mobile references stayed in hover flight at the same position. Additionally, the main diagonal of the P matrix from the Kalman filter is showed in the figure 4.8.

Table 4.3: RMSE of the position estimation

	X [m]	Y [m]	Z [m]
Ascension	0.0203	0.0208	0.0418
Hovering	0.0118	0.0232	0.0342
Translation	0.0298	0.0401	0.0669
All simulation	0.0223	0.0306	0.0448

Table 4.4: Precision of the height estimation performed by the Trilateration Algorithm

Height [m]	Standard Deviation [m]
0	0.0433
0.5	0.0457
1	0.1165

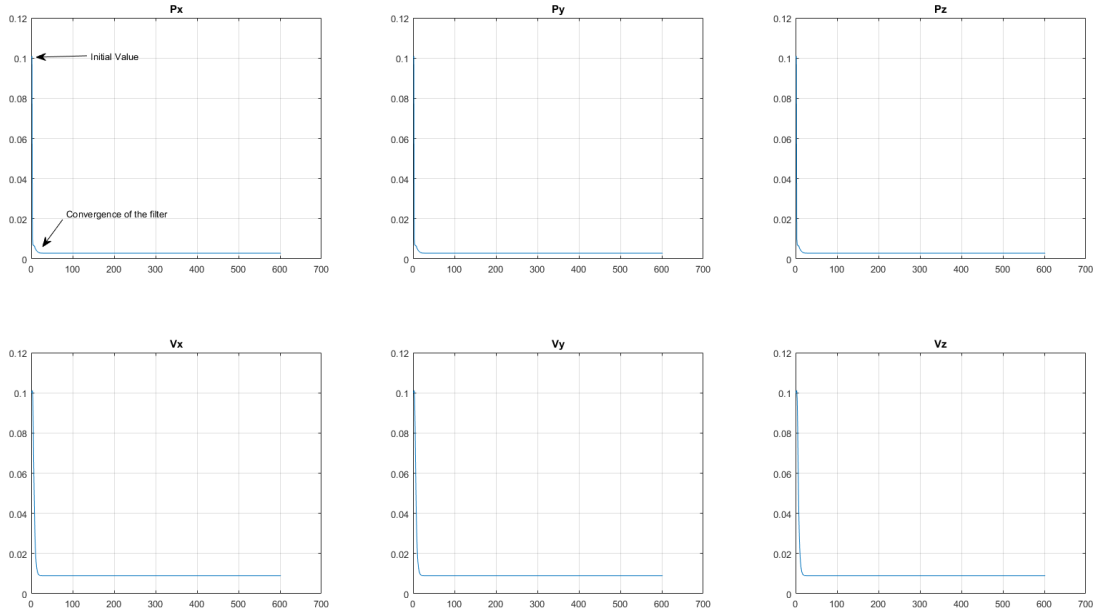


Figure 4.8: P matrix - Kalman filter.

4.2 Flight scenario 2

Two more configurations were considered to evaluate the performance of the system:

1. Translation of the *mobile references* with the *mobile target* stationary in a hover flight (figure 4.9). It was applied a step of 0.5 m in the desired value of X, Y and Z positions of the *mobile references* (horizontal and vertical translations).
2. Translation of the group (figure 4.10). For this second case, the same horizontal/vertical translations were applied to all quadrotors. Therefore, the flight configuration and the relative position between the vehicles should stay the same.

Note that, the mobile target stayed inside the flight area showed in figure 4.3 during the whole flight, for both simulation trials. Thus, the maneuvers performed did not have a significant impact on the position estimation of the system.

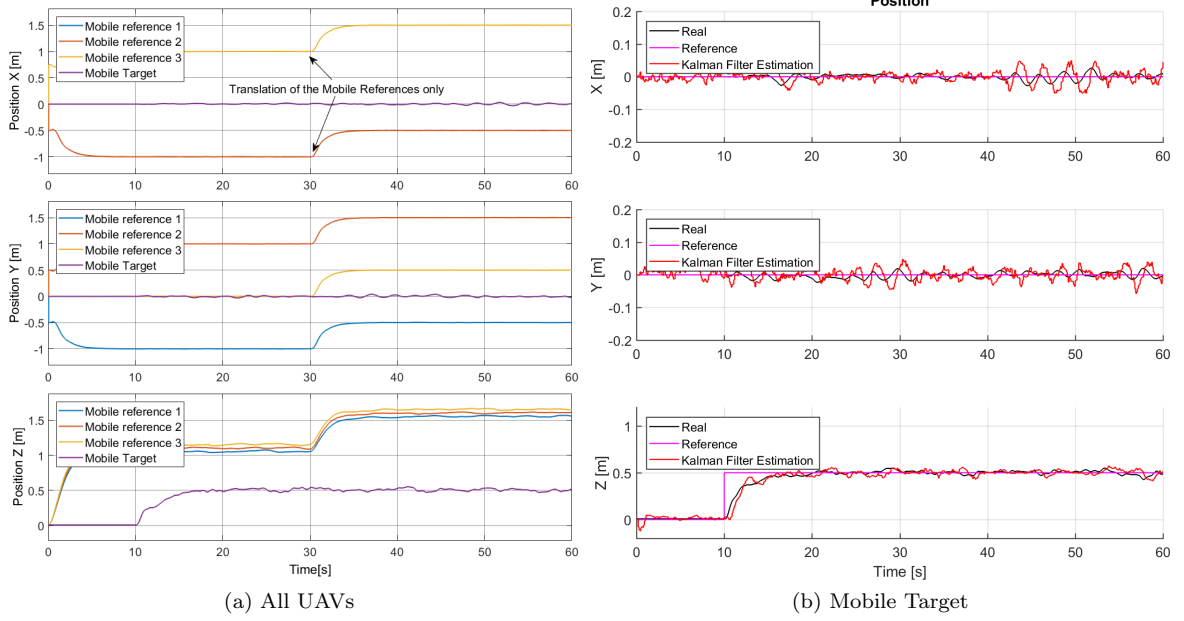


Figure 4.9: Position during the translation of the *mobile references*.

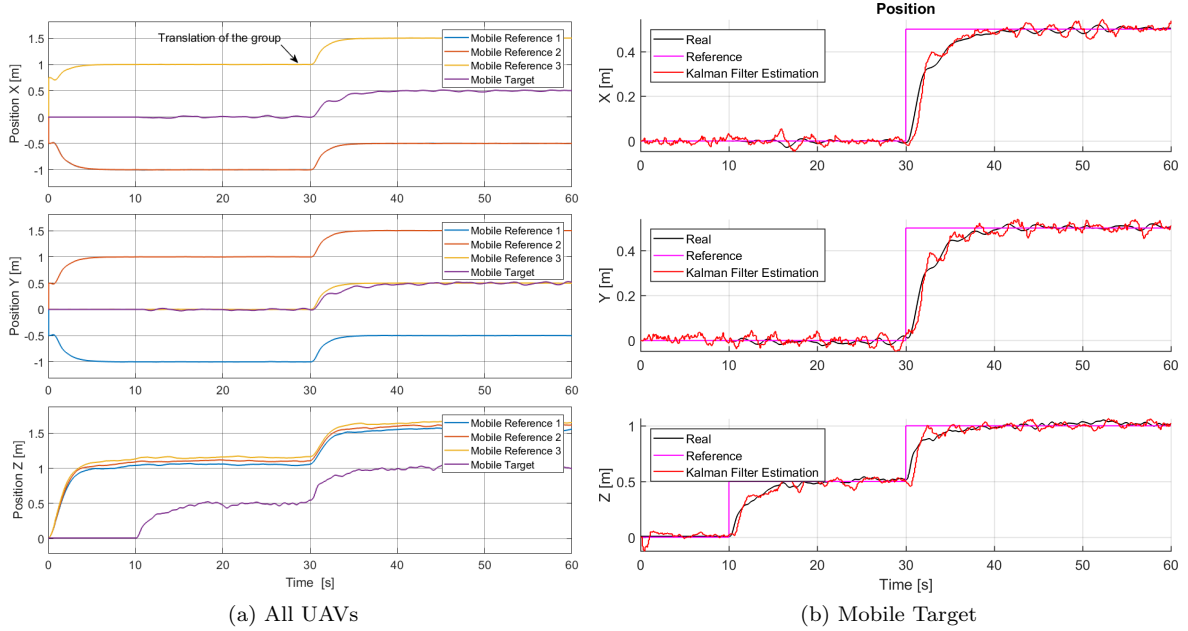


Figure 4.10: Position during the translation of all vehicles.

4.3 Flight scenario 3

Up to this point, the *mobile references* stayed in a same configuration, forming a virtual triangle horizontally, and with a small height displacement (as in table 4.2). Also, the *mobile target* vehicle maneuvered inside this virtual triangle. Comparatively, in this section, it is going to be studied two possible flight configurations: a) flight outside this virtual triangle, and b) different positions for the *mobile references*.

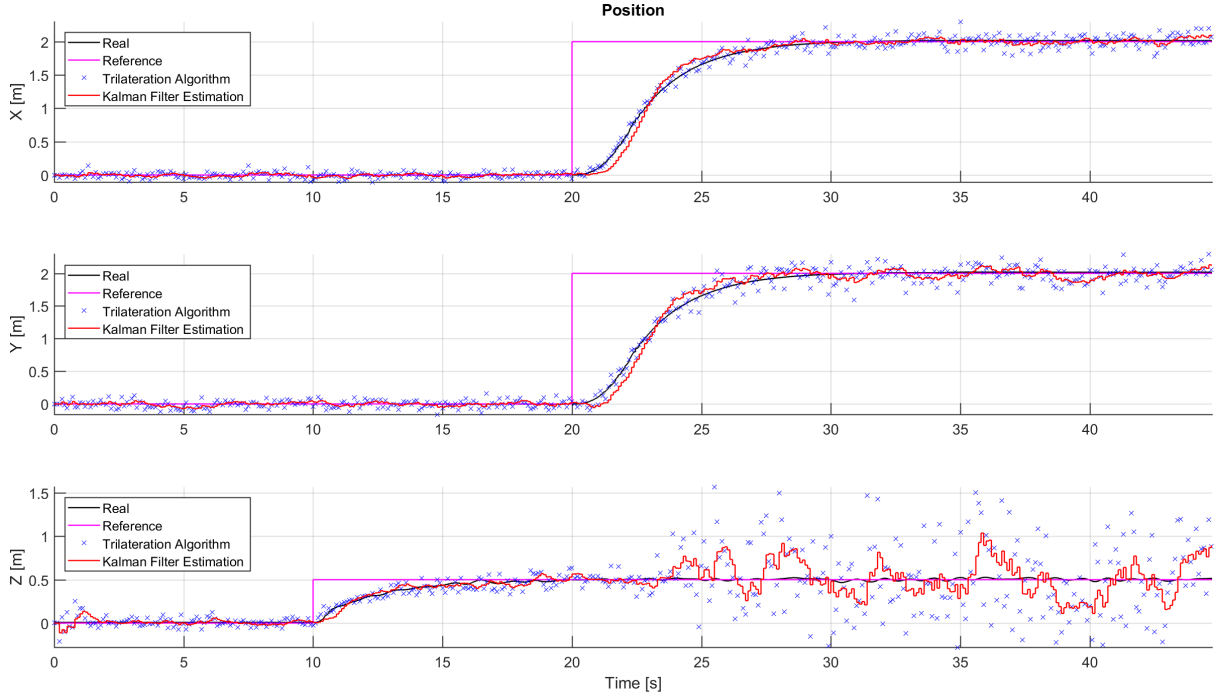


Figure 4.11: Flight outside the flight area.

1. Flight of the *mobile target* out of the flight area (virtual triangle)

During this experiment (figure 4.11), the mobile target started from the middle of the flight area, and moved more than 2 m away from the initial point (outside to the flight area). The estimation error of the overall system (output of the Kalman Filter) increased as the vehicle moved away from the mobile references (tables 4.5 and 4.6). The position controller of the *mobile target* used the ideal value, since the controller could become unstable due to the estimation performance.

Table 4.5: RMSE of the position estimation (output of Kalman filter).

	X [m]	Y [m]	Z [m]
Inside	0.0214	0.0250	0.0390
Outside	0.0334	0.0642	0.1811

Table 4.6: RMSE of the position estimation performed by the trilateration algorithm

	X [m]	Y [m]	Z [m]
Inside	0.0448	0.0595	0.0586
Outside	0.0889	0.1268	0.4221

The overall error of the Kalman Filter estimate is lower than using only the trilateration algorithm. That shows efficiency of the architecture proposed, and how the estimation is positively affected by combining measurements from different sources.

2. Random movement of the *mobile references*

During this experiment, the position of the *mobile references* changed every 15 seconds. It is possible to see in the figure 4.12 the 4 arrangements of the UAVs during the simulation. The *mobile target* maintained a hover flight in the same position, and used the ideal values in the position controller. Note that, for this case, there is no guarantee that the mobile target will flight inside the horizontal virtual

triangle created by the *mobile references*.

The configurations 3 and 4 presented higher estimation error in X and Y positions. This is due to the “bad geometry” of the *mobile references*. A bad geometry is a configuration where the UAVs are not uniformly distributed or are too close to each other, this leads to a position estimation with low accuracy and/or precision.

Figure 4.13 shows the displacement of the quadrotors from a top view, specifically for the configurations 3 and 4, that had a poor performance on the X and Y position estimation. The blue circle represents the *mobile target*, and the red Xs are the *mobile references*.

On the other hand, the best estimation of the height happened for the configuration 4. For that case, the *mobile references* had a height difference of almost 2 m.

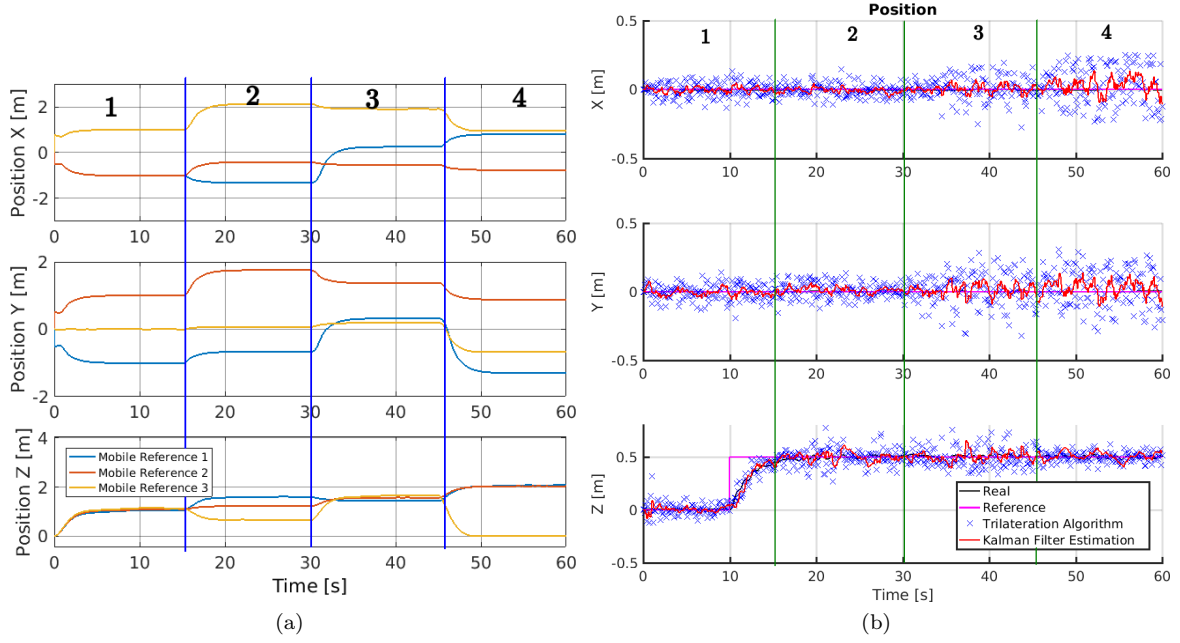


Figure 4.12: Position estimation for different arrangements of the *mobile references*.

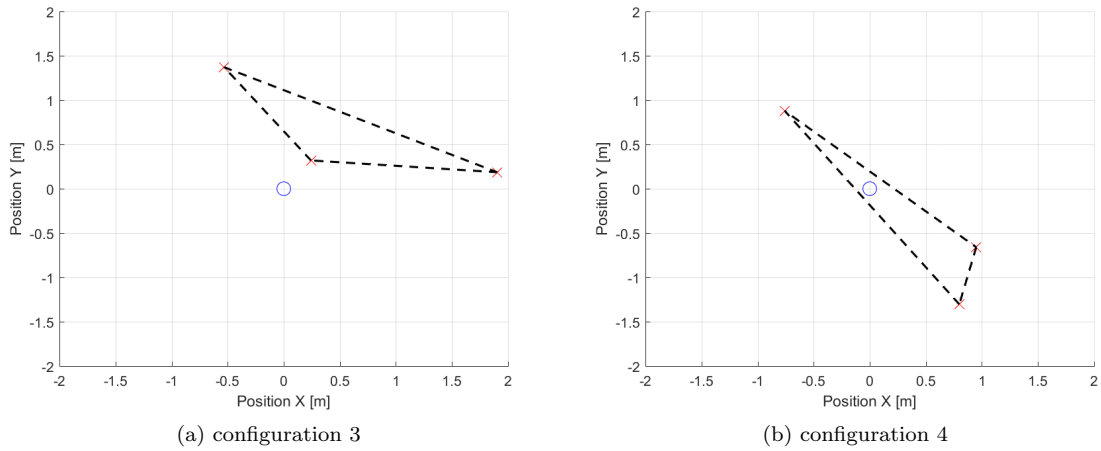


Figure 4.13: Flight configurations.

4.4 Flight scenario 4

Lastly, in this simulation trial it is considered that a height sensor was added to the *mobile target*. Therefore, the height value and the estimated position in X, and Y were added to the trilateration algorithm. Given that, in this section I repeated the the flight scenario 1, but using 4 range measurements instead of 3.

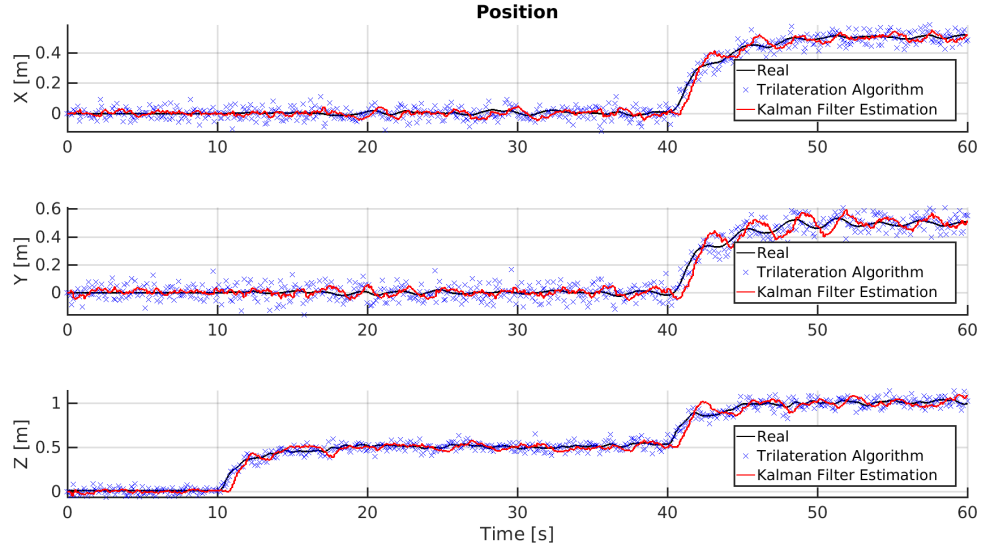


Figure 4.14: Full trajectory of the *mobile target* UAV.

Table 4.7: RMSE of the position estimation of the whole simulation

	X [m]	Y [m]	Z [m]
Trilateration - 3 measurements	0.0250	0.0349	0.0669
Trilateration - 4 measurements	0.0376	0.0536	0.0423
Kalman Filter - 3 measurements	0.0223	0.0306	0.0448
Kalman Filter - 4 measurements	0.0254	0.0387	0.0493

Table 4.8: Standard deviation of the position estimation from the trilateration algorithm

	X [m]	Y [m]	Z [m]
3 measurements	0.0357	0.0452	0.1165
4 measurements	0.0490	0.0656	0.0567

Tables 4.7 and 4.8 compare the estimation performed with 3 range estimations (from flight scenario 1) and 4 range estimations. It is interesting to notice that the Kalman Filter estimation was not weakly affected by the new measurement. On the other hand, the trilateration algorithm was impacted by the addition of the new measurement, specially in the Z calculation. The precision of the height estimation improved, as can be seen by the standard deviation value on table 4.8.

Chapter 5

Conclusion

During this work, I proposed a system architecture for position estimation of a quadrotor using Cooperative Localization approach. In order to make the approach feasible in a short time, some simplifications were made, e.g., by assuming that the *mobile references* had a ideal position information.

From the simulation trials performed, it was possible to identify that the system is suitable to perform the position estimation of the UAV. I also presented some examples of “bad geometry”, where the displacement of the UAVs caused the estimation to have a poor performance.

It was possible to see that the trilateration algorithm is directly affected by the displacement of the UAVs in the environment. However, it was also showed that the integration of measurements using Kalman Filter was robust, and the final position estimation was satisfactory even for a certain level of uncertainty in the trilateration computation.

All things considered, a further study of the system architecture is needed before implementing this solution in real vehicles. In a future work, I would like to study a few topics, such as: a) the effect of the attitude errors in the position estimation, or b) the cross correlation between the estimations performed by each quadrotor.

Bibliography

- [1] F. Bounini, D. Gingras, H. Pollart, and D. Gruyer, “Real time cooperative localization for autonomous vehicles,” in *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*, pp. 1186–1191, Nov 2016.
- [2] R. Kurazume, S. Nagata, and S. Hirose, “Cooperative positioning with multiple robots,” in *ICRA*, vol. 2, pp. 1250–1257, 1994.
- [3] S. Saeedi, M. Seto, and H. Li, “Fast monte carlo localization of auv using acoustic range measurement,” in *2015 IEEE 28th Canadian Conference on Electrical and Computer Engineering (CCECE)*, pp. 326–331, IEEE, 2015.
- [4] A. Bahr, J. J. Leonard, and M. F. Fallon, “Cooperative localization for autonomous underwater vehicles,” *The International Journal of Robotics Research*, vol. 28, no. 6, pp. 714–728, 2009.
- [5] S. Tanwar and G. X. Gao, “Decentralized collaborative localization with deep gps coupling for uavs,” in *Position, Location and Navigation Symposium (PLANS), 2018 IEEE/ION*, pp. 767–774, IEEE, 2018.
- [6] G. Shen, R. Zetik, and R. S. Thoma, “Performance comparison of toa and tdoa based location estimation algorithms in los environment,” in *Positioning, Navigation and Communication, 2008. WPNC 2008. 5th Workshop on*, pp. 71–78, IEEE, 2008.
- [7] A. Gomes, C. Rubens, S. Braley, and R. Vertegaal, “Bitdrones: Towards using 3d nanocopter displays as interactive self-levitating programmable matter,” in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, CHI ’16, (New York, NY, USA), pp. 770–780, ACM, 2016.